Will it print count

```
function Counter () {
   var count = 0;
   console.log("count:",count)
   // return  count;
   this.incrementCounter = function () {
      count++;
      console.log(count);
   }

   this.decrementCounter = function () {
      count--;
      console.log(count);
   }
}


var counter1  = new Counter();


will it print console count or not?
```

Yes ✅ — it **will print the `console.log("count:", count)` statement** once when you create a new instance using:

```js
var counter1 = new Counter();
```

Step-by-step explanation 👇

1. When you call **`new Counter()`**,
   JavaScript does the following behind the scenes:
   - Creates a new empty object → **`{}`**
   - Sets the prototype of that object to **`Counter.prototype`**
   - Sets **`this`** inside the **`Counter`** function to that new object
   - Executes the **`Counter()`** function body
   - Returns the new object (unless the function explicitly returns another object)

---

2. Inside your function:

```
js

function Counter () {
    var count = 0;
    console.log("count:", count);
    // ...
}
```

- The **`var count = 0;`** line creates a **local variable** (private to this function).
- Then **`console.log("count:", count)`** runs immediately when the constructor is executed.

So when you call:

```
js

var counter1 = new Counter();
```

you will see:

```
makefile

count: 0
```

printed in the console.

---

3. Afterwards, you can call:

```
js

counter1.incrementCounter(); // prints 1
counter1.decrementCounter(); // prints 0
```

Summary:

✅ Yes, it prints **`count: 0` once when the constructor runs.**
Because that **`console.log()`** is executed during the function call made by **`new Counter()`**.